

Modificabilidad (Modifiability)

Modificabilidad (Modifiability)

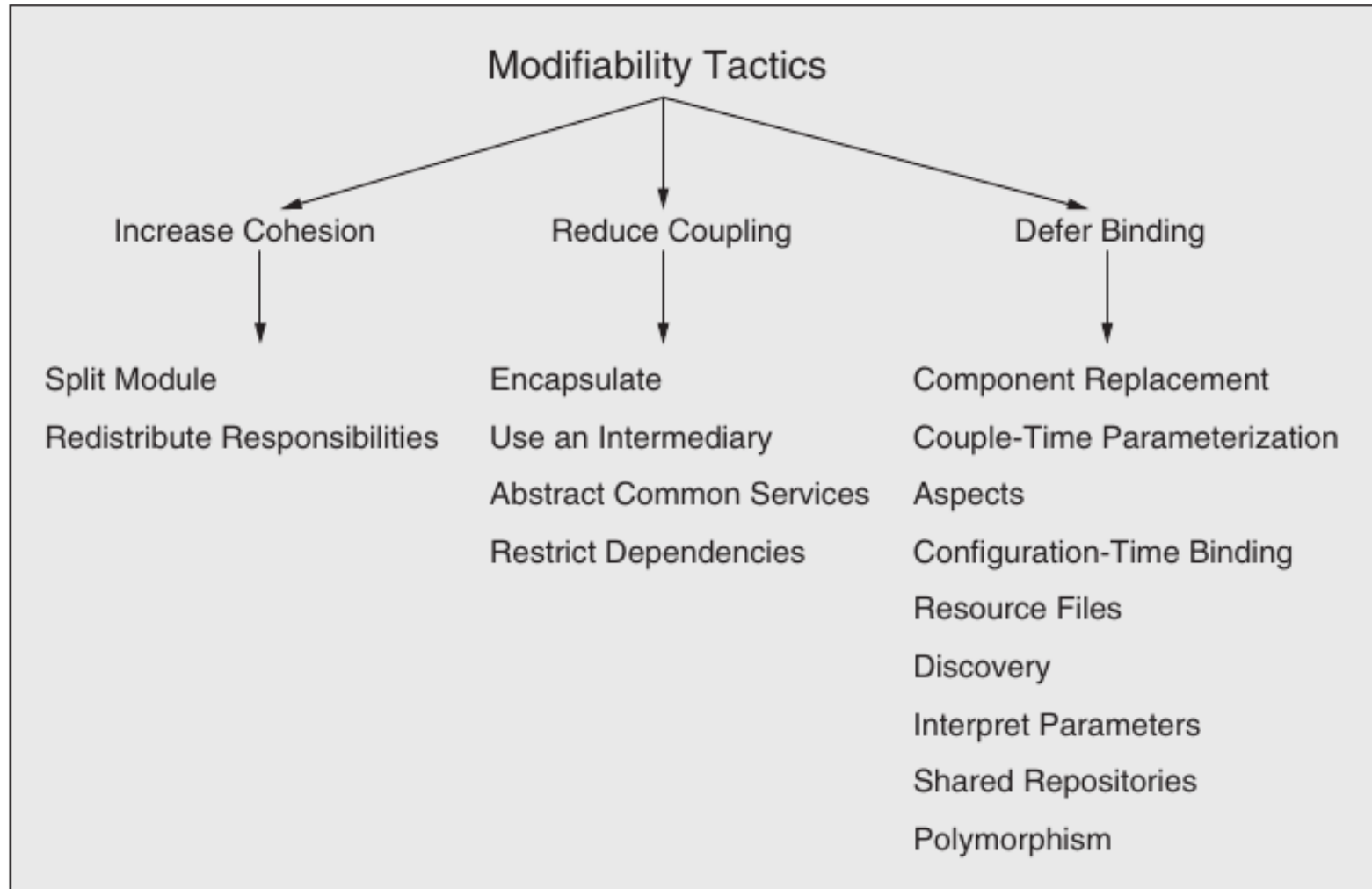
- ¿Qué puede cambiar?
 - Funciones
 - Plataforma
 - Entorno (otros sistemas)
 - Otros atributos de calidad
- ¿Qué tan probable es el cambio?
- ¿Quiénes pueden hacer el cambio?
 - Desarrolladores
 - Administradores
 - Usuarios finales
- ¿En qué momento?
 - En desarrollo
 - En producción
- ¿Cuánto cuesta?
 - Sistema flexible, pero costoso de desarrollar
 - Sistema que requiere cambios de los desarrolladores, pero es menos costoso

Atributos relacionados

- Escalabilidad
- Variabilidad
 - Capacidad del sistema de crear variaciones a la **funcionalidad** original
 - Ej.: Variación por tipos de licencia, líneas de producto
- Portabilidad
 - Capacidad de desplegarse en diferentes plataformas
- Independencia de ubicación
 - Capacidad de dos partes de un sistema de interactuar independientemente de dónde estén desplegados

Tácticas

Tácticas



Tácticas para incrementar cohesión

Recordemos: ¿Qué es cohesión?



Dividir módulos

- Un módulo grande es probable que tenga baja cohesión
- Descomponer por funcionalidades conceptualmente cercanas

Redistribuir responsabilidades

- Buscar funcionalidades conceptualmente cercanas, pero dispersas en el código
- Integrarlas en un componente independiente

Tácticas para reducir acoplamiento

Encapsular

- A nivel de un componente individual
 - Definir una interfaz para cada componente
 - Ocultar los elementos internos del componente
 - Solo exponer lo esencial
 - Toda la interacción con ese componente **debe** pasar por la interfaz
- A nivel de varios componentes
 - Exponer solo las interfaces de componentes que sean relevantes
 - Ocultar interfaces de componentes internos que no sean directamente necesarios

Usar intermediarios (Indirection)

- Para grupos de componentes
 - Crear un componente que actúe como intermediario con el exterior
 - Todas las peticiones a dicho grupo **deben** pasar por dicho componente intermediario
- Patrones relacionados
 - Controller
 - Facade
- Ejemplos
 - Colas de mensajes
 - Servicios en general

Abstraer servicios comunes

- Crear una interfaz abstracta común para dos o más servicios similares
 - Complementarla con un Intermediarios
- Ejemplo:
 - JDBC
 - Hardware Abstraction Layers (HAL)
 - Drivers de computadores
 - Servicios de pago online

Restringir dependencias

- Después de Encapsular...
- Restringir
 - Qué grupos de componentes se pueden comunicar entre sí
 - En qué dirección se pueden comunicar
- Ejemplo
 - Restricciones de modelo N-Tier
 - Solo se puede comunicar con capas inferiores
 - Diferencia con Intermediario
 - Son restricciones que normalmente no se pueden controlar desde el lenguaje de programación

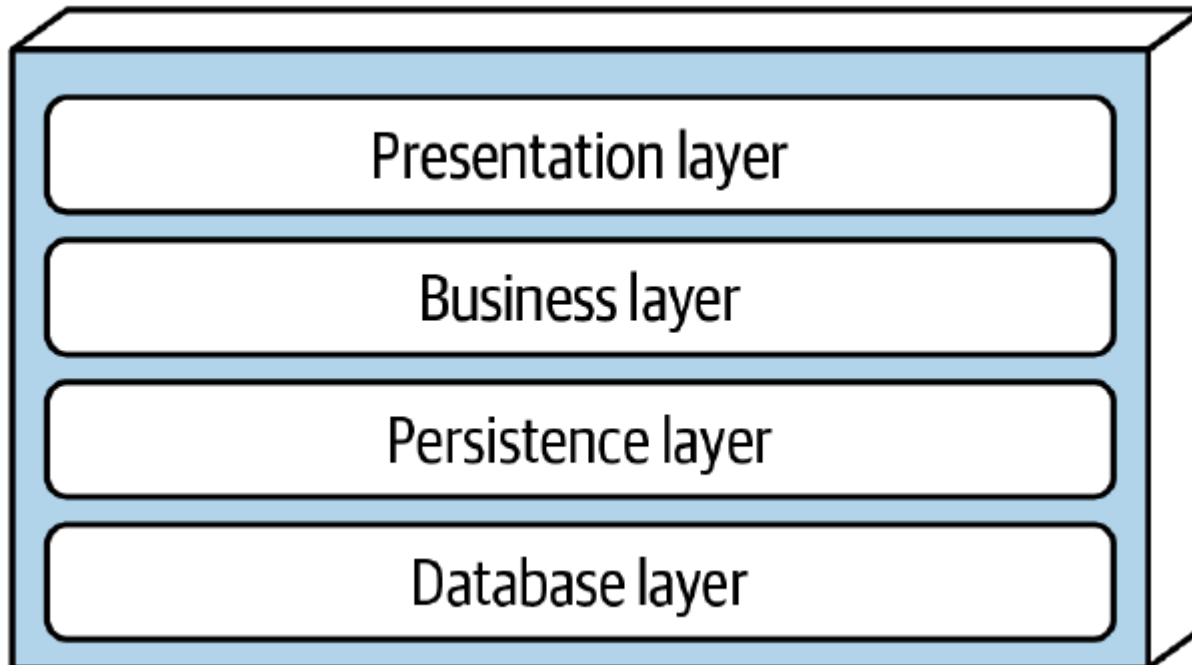
Postergar Enlace (Defer Binding)

- Binding: Conectar una interfaz con implementación
- Postergar enlace:
 - Flexibilizar cómo y cuándo la interfaz se conecta a una implementación
 - En algunos casos, desacoplar interfaz de implementación
- Tácticas específicas
 - Binding en tiempo de compilación
 - Ej: [configure + Makefile](#)
 - Binding en tiempo de ejecución
 - Service Discovery
 - Ej: Spring Cloud Eureka
 - Interpretar parámetros (archivos de configuración)
 - Ej: [Spring Boot application properties](#), [Command-line Arguments in Spring Boot](#)
 - Repositorios compartidos / colas de mensajes
 - Ej:
 - » Redis, etcd (Kubernetes) para guardar configuración centralizadamente
 - » Kafka
 - Polimorfismo

Patrones de Modificabilidad

- Todos los estilos arquitectónicos aportan a Modificabilidad
- Nos enfocaremos en los más evidentes

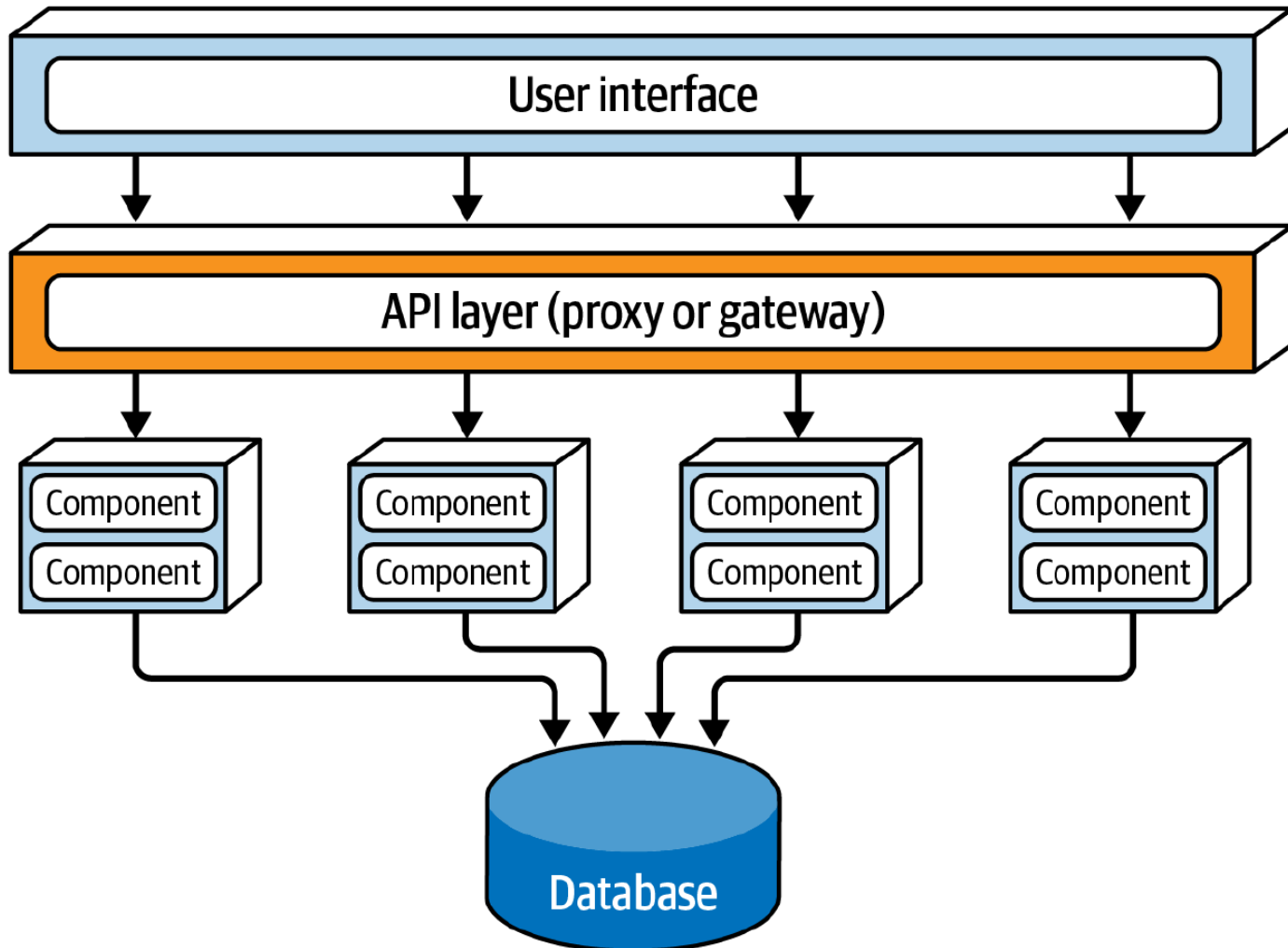
Layered / N-Tier



Layered / N-Tier

- Sistema se divide en N capas
- Cada capa tiene responsabilidades claramente definidas
- Cada capa solo debería comunicarse con la capa inmediatamente inferior
 - Recomendable evitar bridging (saltarse capas)
- Ventajas
 - Reduce acoplamiento
 - Facilita sustituir capas (interfaces claramente definidas)
- Desventajas
 - Overhead de comunicación entre capas (incluso si es monolito)
 - Podría ocultar funciones de bajo nivel útiles para capas superiores
 - Ej. ORM vs SQL
 - Bridging puede dificultar mantenimiento

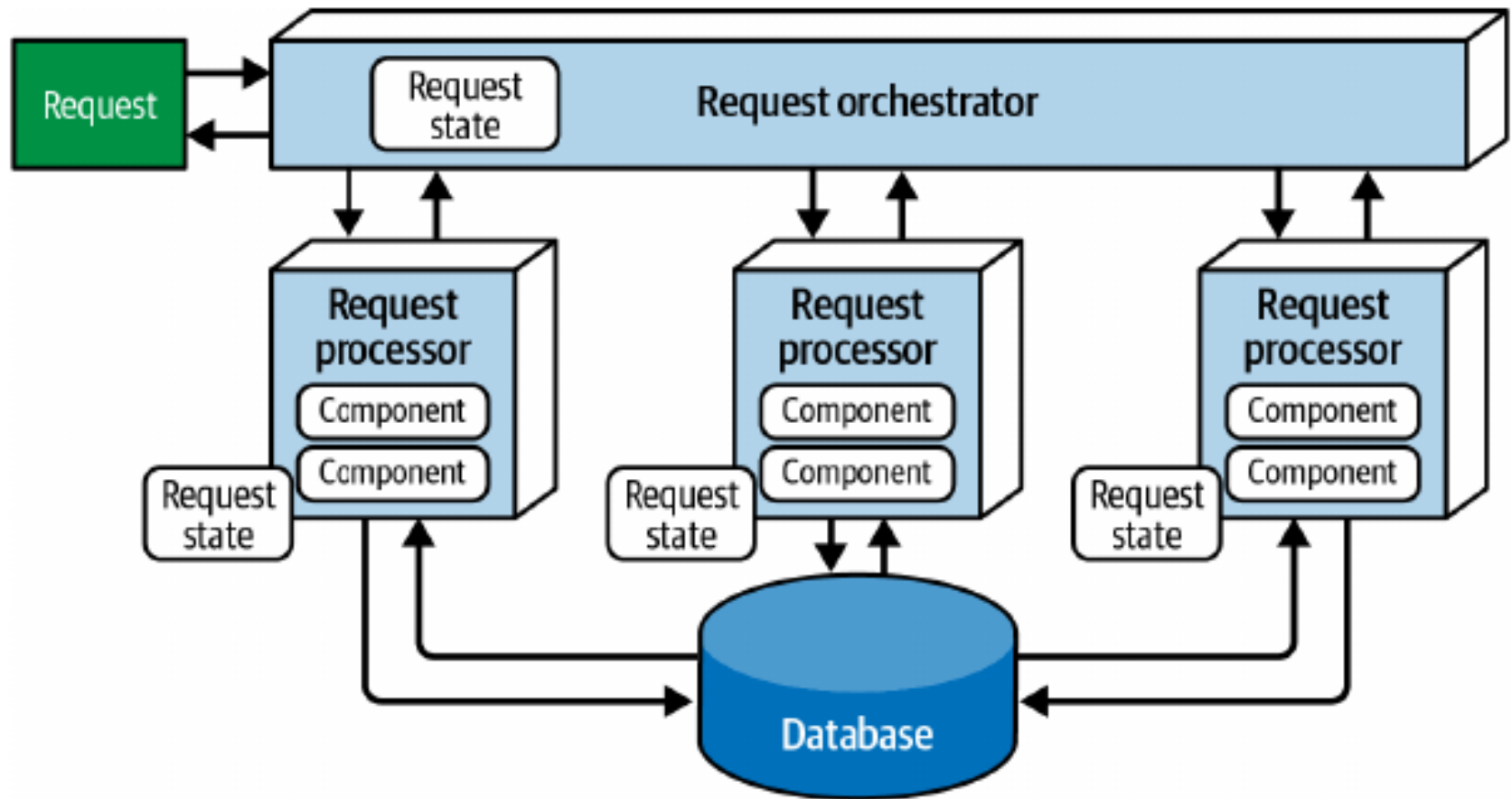
Service-based



Service based

- Servicios
 - “Gruesos”
 - Relativamente independientes
 - Desplegables independientemente
 - Pueden tener replicación y balanceo de carga
- Base de datos
 - BD centralizada (lo mas común)
 - Una BD por servicio
 - Múltiples BD compartidas entre algunos servicios
- Recomendable un único punto de entrada
 - Una o múltiples UI se conecta por red
- Se complementa con Layered/N-Tier

Event-driven Architecture (EDA) / Message-driven Architecture (MDA)

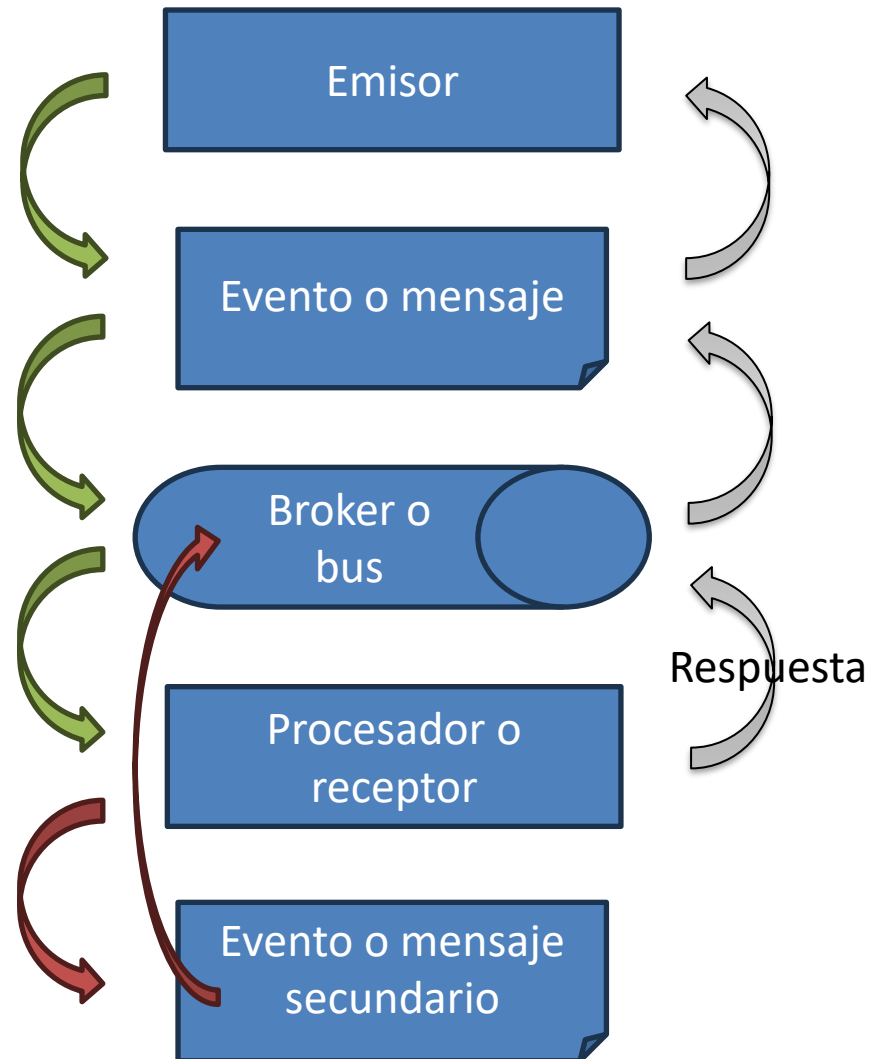


Event-driven Architecture (EDA) / Message-driven Architecture (MDA)

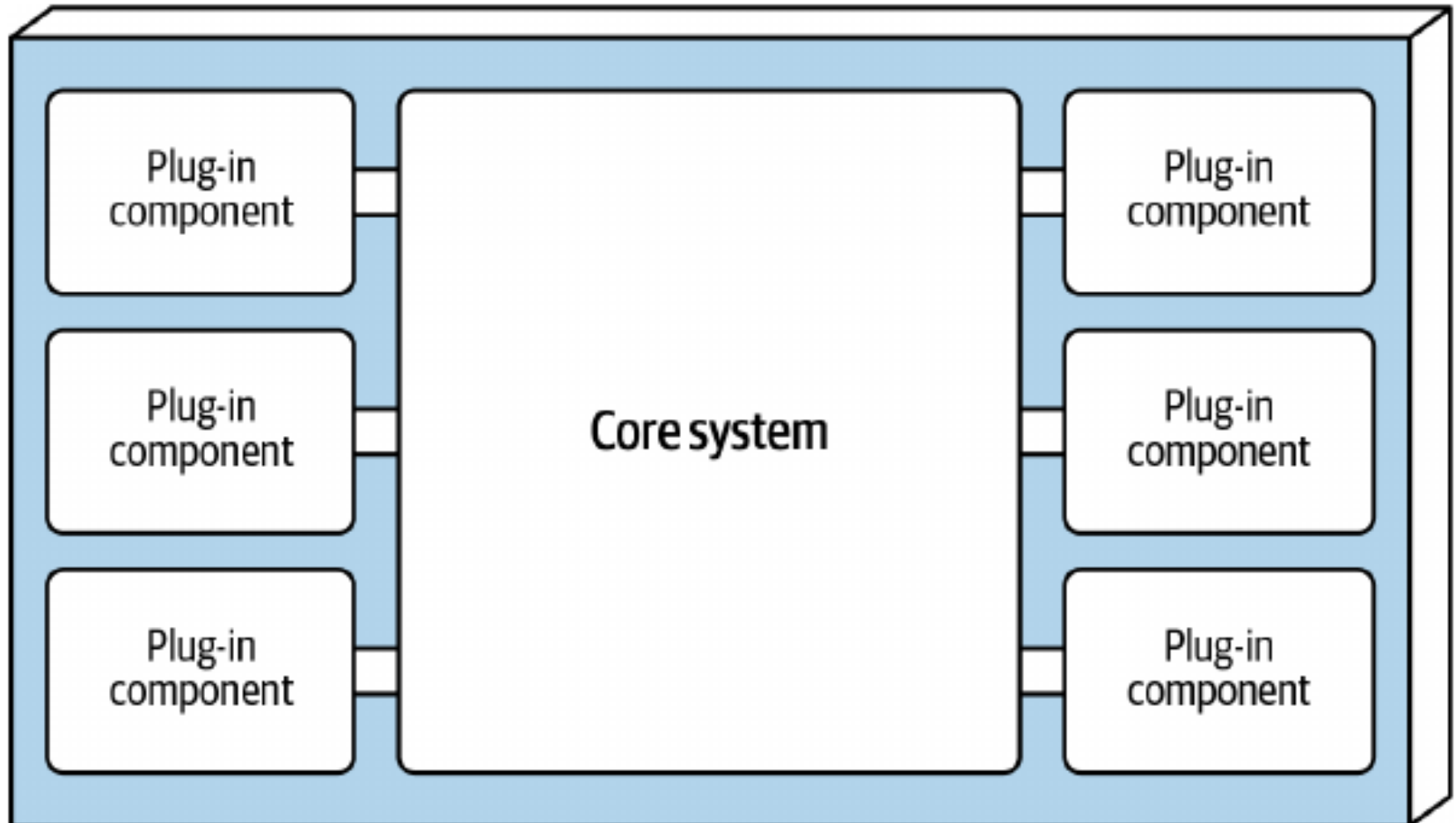
- Usar colas o publish-subscribe para enviar eventos y mensajes, ambos asíncronos
- Eventos
 - Describen algo que sucede
 - Unidireccionales (fire-and-forget)
 - Múltiples consumidores
- Mensajes
 - Describen una orden que otro componente debe atender
 - Requieren respuesta
 - Típicamente un solo consumidor

Componentes de EDA/MDA

- Emisor
- Evento o mensaje inicial
- Broker o Bus de eventos/mensajes
- Procesador o receptor
- Eventos o mensajes derivados



Microkernel



Microkernel

- También llamada “Plug-in architecture”
- Núcleo con funcionalidades minimas.
- Funcionalidades adicionales (Plugins) se instalan después.
 - Normalmente lo hace el usuario final
- Ejemplos
 - Sistema operativo. Ej. QNX
 - Aplicación con funcionalidad base. Ej. IDE, Navegadores
 - Algunas Apps web. Ej Trello, Jira
 - Videojuegos. Ejemplos: [Minecraft](#)